IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

Yenepoya Arts, Science, Commerce and Management

Final Project Report

on

Natural Language Interface for E-commerce infrastructure

Student Name: Abhishek.k.k

Industry Mentor: Mr.Shashank **Guided by: Ms.Yuktha**

Table of Contents

1. Executive Summary 3
2. Background 4
    2.1 Aim 6
    2.2 Technologies 7
    2.3 Hardware Architecture 9
    2.4 Software Architecture 12
3. System 13
    3.1 Requirements 15
        3.1.1 Functional Requirements 15
        3.1.2 User Requirements 16
        3.1.3 Environmental Requirements 17
    3.2 Design and Architecture 18
    3.3 Implementation 21
    3.4 Testing 23
    3.5 Graphical User Interface (GUI) Layout 25
    3.6 Customer Testing 28
    3.7 Evaluation 30
4. Snapshots of the Project 34
5. Conclusions 38
6. Further Development or Research 40
7. References 42
8. Appendix 43
9. Weekly report 48

1.Executive Summary

The Natural Language Interface for E-commerce Infrastructure is a system designed to provide users with an efficient and flexible way to interact with an e-commerce platform. The system supports both a Command Line Interface (CLI) and a Web-based Single Page Application (SPA), allowing users to perform operations such as browsing products, adding items to the cart, and completing checkout.

The project demonstrates a dual-interface architecture, where both CLI and web clients communicate with a centralized backend using REST APIs. This approach ensures that the system is modular, scalable, and easy to maintain. The backend is developed using the Flask framework in Python, which handles all business logic and processes requests from the frontend.

The system uses a JSON file as the data storage mechanism, where product information and cart details are stored. The CLI interface focuses on simplicity and supports menu-driven interaction, while the web interface provides a more interactive user experience with features such as authentication, Wishlist management, and order history.

The web frontend also uses local Storage to maintain client-side state, improving performance and responsiveness. Overall, the system highlights key software engineering concepts such as RESTful communication, separation of concerns, and efficient data handling.

Overall, the project demonstrates key software engineering concepts such as modular design, API communication, and separation of concerns. It provides a clear understanding of how an e-commerce system works at a fundamental level and serves as a strong foundation for future enhancements like database integration, graphical user interfaces, and advanced features.

2. Background

The rapid growth of information technology has transformed business operations, especially in commerce. Traditional buying and selling methods have largely been replaced by e-commerce systems, which allow users to browse products, compare prices, and complete transactions online without time or location limits. Most e-commerce platforms use web or mobile applications with graphical interfaces, but command-line interface (CLI) systems are also important in development, testing, and lightweight environments.

A CLI is a text-based system where users interact with software by entering commands in a terminal. Unlike graphical interfaces, it does not use buttons or icons, but instead focuses on speed, efficiency, and low resource usage. Although it may be less user-friendly for beginners, CLI systems are widely used by developers for automation and backend operations due to their flexibility and control.

This project focuses on the development of a Natural Language Interface for E-commerce Infrastructure, which supports both a CLI-based system and a web-based Single Page Application (SPA). The system allows users to perform essential e-commerce operations such as viewing products, adding items to a cart, and completing checkout. The inclusion of both CLI and web interfaces demonstrates a dual-interface architecture, providing flexibility for different types of users.

The system follows a client-server architecture, where both the CLI and web frontend act as clients, and a Flask-based backend handles business logic and data processing. The backend provides RESTful APIs such as retrieving products, managing cart operations, and processing checkout requests. Communication between the client and server is achieved through HTTP requests and JSON responses, ensuring a consistent and modular design.

For data storage, a JSON file (data. Json) is used instead of a traditional database. It stores product details and cart information, making the system simple and lightweight. In addition, the web frontend maintains client-side state using local Storage, which stores user session data, cart

details, Wishlist, and order history. While the JSON-based approach simplifies implementation, it has limitations in terms of scalability and concurrency.

Overall, this system demonstrates the core concepts of modern e-commerce applications in a simplified yet effective manner. It provides a strong foundation for future enhancements such as database integration, advanced authentication mechanisms, graphical improvements, and multi-user support.

2.1 Aim

The primary aim of this project is to design and develop a functional E-commerce system using a command-line interface, demonstrating key concepts of modern software development such as client-server architecture, RESTful APIs, and data persistence. The system provides a simplified model of an online shopping platform where users can perform basic operations like viewing products, adding items to a cart, and completing purchases efficiently.

The project focuses on providing a dual-interface system, consisting of both a Command Line Interface (CLI) and a Web-based Single Page Application (SPA). The CLI ensures simplicity and lightweight interaction through a menu-driven approach, while the web interface provides a more interactive and user-friendly experience with additional features such as authentication, Wishlist, and order history.

Another important aim is to implement a client-server architecture using Flask as the backend. Both the CLI and web frontend act as clients, sending HTTP requests to the server, which processes them and returns JSON responses. This separation improves modularity, scalability, and maintainability of the system.

The system also demonstrates RESTful API communication, where different HTTP methods are used for operations such as retrieving products, managing the cart, and processing checkout. This ensures clear and standardized communication between components.

Data persistence is achieved using a JSON file (data.json), which stores product details and cart information. Additionally, the web frontend maintains client-side state using local Storage, enabling storage of user session data, cart items, Wishlist, and order history. This improves performance and enhances user experience.

Finally, the project aims to provide a practical understanding of real-world software development by covering the complete lifecycle of a system, including design, implementation, testing, and evaluation. It serves as a foundation for building more advanced and scalable e-commerce applications.

2.2 Technologies

The development of the E-commerce system is based on a combination of modern and lightweight technologies selected for their simplicity, efficiency, and suitability for building a modular application. Each technology plays a specific role in ensuring smooth communication between system components.

The primary programming language used is Python, which is known for its simplicity, readability, and extensive library support. Python is used for backend development as well as for implementing the CLI interface, ensuring consistency across the system.

The backend is developed using the Flask framework, a lightweight web framework in Python. Flask is used to create RESTful APIs that handle client requests such as retrieving products, managing cart operations, and processing checkout. Its simplicity and flexibility make it suitable for small to medium-scale applications.

The system provides two types of frontends. The first is a Command Line Interface (CLI) developed in Python, which allows users to interact through a menu-driven system. The second is a Web-based Single Page Application (SPA) developed using HTML, CSS, and JavaScript, providing a more interactive and user-friendly interface.

For communication between frontend and backend, the system uses the HTTP protocol with REST APIs. The CLI uses the requests library in Python, while the web frontend uses the Fetch API in JavaScript to send GET and POST requests and receive JSON responses.

For data storage, the system uses a JSON file (data.json), which stores product details and cart information. This provides a simple and lightweight form of data persistence. Additionally, the web frontend uses localStorage to maintain client-side data such as user session, cart items, wishlist, and order history

The built-in json module in Python is used to handle JSON data, enabling easy conversion between Python objects and JSON format.

The development environment includes Visual Studio Code (VS Code), which provides features such as debugging, syntax highlighting, and an integrated terminal, improving development efficiency.

In conclusion, the combination of Python, Flask, CLI, web technologies, JSON storage, and RESTful communication forms a simple yet effective technology stack. These technologies ensure that the system is modular, efficient, and easy to maintain.

2.3 Hardware Architecture

The hardware architecture of the E-commerce system defines the physical components and computing environment required to run the application efficiently. This system is designed as a lightweight application and does not require complex infrastructure. It operates on a single-machine setup, making it suitable for development, testing, and small-scale usage.

The system follows a single-node architecture, where both the frontend and backend components run on the same computer. The frontend includes both the Command-Line Interface (CLI) and the Web-based interface, while the backend is implemented using Flask. All communication between these components takes place within the same machine through the localhost network interface.

The primary hardware requirement for this system is a standard personal computer or laptop. The minimum configuration required includes a processor such as Intel i3 or its equivalent, along with at least 4GB of RAM. Since the system performs basic operations such as handling user input, processing requests, and reading or writing data to a file, it does not demand high computational power. However, a higher configuration can improve responsiveness and overall performance.

Memory plays an important role in ensuring smooth operation. The system loads product and cart data from a JSON file into memory during execution. Adequate RAM ensures that these operations are carried out efficiently without delays. Since the data size in this project is relatively small, the memory requirement remains minimal.

Storage is another essential component of the hardware architecture. The system uses a JSON file to store product information and cart details. This file is stored locally on the system's storage device, such as a hard disk drive (HDD) or solid-state drive (SSD). The storage requirement is very low, usually only a few megabytes. However, faster storage devices like SSDs can improve the speed of file access operations.

Input and output devices are also part of the system. The primary input device is the keyboard, which is used to interact with the CLI interface. For the web interface, users interact through both keyboard and mouse. The output is displayed either in the terminal (for CLI) or in a web browser (for the web interface). Since the system does not use graphical processing, it does not require a dedicated graphics card.

The system runs on a standard operating system such as Windows, Linux, or macOS. The operating system manages system resources like CPU usage, memory allocation, and file handling. It also provides the environment required to run Python and the Flask server

Communication between frontend and backend occurs through the localhost network interface (127.0.0.1) using a specific port, typically port 5000. This loopback interface allows the system to simulate client-server communication without requiring an external network connection. As a result, the communication is fast, secure, and reliable.

The overall hardware interaction can be represented as:

User Input (Keyboard/Mouse) → Frontend (CLI/Web) → Localhost Network → Flask Backend → JSON Storage → Response → Output (Terminal/Browser)

In a real-world scenario, the hardware architecture would be more complex. The backend would run on a dedicated server or cloud platform, and users would access the system through the internet. Additional components such as database servers, load balancers, and network infrastructure would be required to support multiple users and large-scale operations.

In conclusion, the hardware architecture of the E-commerce system is simple, efficient, and suitable for its intended purpose. It provides a clear understanding of how software interacts with hardware components while maintaining low resource requirements. Although it is not designed for large-scale deployment, it serves as a strong foundation for future enhancements and scalability.

2.4 Software architecture

Software architecture defines the overall structure of a system and how its components interact with each other. The E-commerce CLI system is designed using a simple, modular, and maintainable architecture that reflects real-world application design principles.

The system follows a three-tier architecture, which includes the presentation layer, application layer, and data layer. This separation of layers ensures better organization, easier maintenance, and independent development of each component.

The presentation layer is responsible for user interaction. This system includes two types of interfaces:

- A Command-Line Interface (CLI) developed in Python
- A Web-based Single Page Application (SPA) developed using HTML, CSS, and JavaScript

The CLI provides a menu-driven interface where users can perform operations such as viewing products, adding items to the cart, and checking out. It also supports simple text commands for flexibility.

The application layer (business logic layer) is built using the Flask framework. It processes user requests, performs validations, and executes core operations. Flask exposes RESTful API endpoints for different functionalities such as product retrieval, cart management, and checkout. It also ensures rules like stock validation and proper request handling.

The data layer is responsible for storing system data. In this project, a JSON file is used as a lightweight database to store product details and cart information. The backend reads data from the file when processing requests and updates it when changes occur. This ensures data persistence. In addition, the web interface uses local Storage to store client-side data such as cart items and user session information. This reduces server dependency and improves performance.

Although JSON storage is simple and efficient, it has limitations in scalability and is suitable mainly for small-scale applications.

Client-Server Communication

The system follows a client-server architecture, where both CLI and web interfaces act as clients, and the Flask application acts as the server.

Communication is done using the HTTP protocol, with:

- CLI using the *requests* library
- Web interface using the *Fetch API*

All data is exchanged in JSON format, ensuring structured and consistent communication.

Data Flow and Design

The overall data flow in the system is:

User Input → Frontend → HTTP Request → Backend → JSON Data → Response → Display Output

This structured flow ensures that all operations are processed efficiently and consistently.The system is designed using a modular approach, where each component handles a specific function. This improves code readability, simplifies debugging, and allows easy addition of new features.

Conclusion: The software architecture of the E-commerce system is simple yet effective. By using a three-tier structure, client-server model, and modular design, the system achieves a balance between functionality and simplicity.

3. System

The system developed in this project is an E-commerce application designed to simulate the core functionalities of an online shopping platform. It allows users to browse products, manage a shopping cart, and complete the checkout process. The system is implemented using both a Command-Line Interface (CLI) and a Web-based interface, providing flexibility in user interaction.

The system follows a client-server architecture, where the frontend interfaces (CLI and Web) act as clients and communicate with a backend server developed using the Flask framework. This separation of components ensures modularity and makes the system easier to maintain and extend.

System Components

The system consists of three main components:

1. CLI Frontend

The CLI frontend is implemented in Python and provides a menu-driven interface. Users can interact with the system by entering numeric options or simple text commands. It is lightweight and efficient, making it suitable for development and testing environments.

2. Web Frontend (SPA)

The web frontend is developed using HTML, CSS, and JavaScript as a Single Page Application (SPA). It provides a more interactive and user-friendly experience. Users can view products, manage their cart, and perform checkout through a graphical interface.

3. Flask Backend

The backend is implemented using Flask and acts as the central processing unit of the system. It handles all business logic, processes client requests, and interacts with the data storage layer. The backend exposes RESTful APIs that both CLI and web interfaces use.

System Workflow

The workflow begins when a user interacts with either the CLI or the web interface. The user selects an action such as viewing products or adding items to the cart. The frontend captures this input and sends an HTTP request to the backend.

The backend receives the request, validates the input, and performs the required operation. For example, when a user requests product details, the backend retrieves the data from the JSON file and sends it back as a response. When adding items to the cart, the backend checks product availability, updates the cart, and modifies stock values.

The response is then returned to the frontend, which displays the result to the user in a structured format.

Data Management

The system uses a JSON file as a lightweight data storage mechanism. It stores product information and cart details. The backend reads data from this file when processing requests and writes updates back to ensure persistence.

Features of the System

The system provides the following key features:

Displaying product catalogue with details

Adding items to the shopping cart

Viewing cart contents and total cost

Performing checkout

Supporting both CLI and web-based interaction

Conclusion: The E-commerce system provides a simplified but effective model of an online shopping platform. By combining CLI and web interfaces with a Flask backend, it demonstrates key concepts such as client-server communication, RESTful APIs, and data management.

3.1 Requirements

The requirements of a system define what the application is expected to do and the conditions under which it should operate. They act as a foundation for system design, implementation, and testing. In this project, the requirements of the E-commerce system are categorized into functional requirements, user requirements, and environmental requirements.

3.1.1 Functional Requirement

Functional requirements describe the core operations of the system. The system must allow users to view a list of available products along with details such as name, price, and stock availability. Users should be able to select products and add them to a shopping cart by specifying the required quantity.

The system must provide a feature to view cart contents, including selected items, quantities, and total cost. It should also support a checkout process where users can finalize their purchase, after which the cart is cleared and a confirmation message is displayed.

Input validation is an important requirement. The system must ensure that user inputs are valid, such as checking for correct product selection and available stock. In case of invalid input or errors, the system should display appropriate messages.

The system must also support communication between frontend and backend using RESTful APIs. Both the CLI and web interfaces should send requests and receive responses in JSON format.

3.1.2 User Requirements

User requirements focus on usability and user experience. The system should provide a simple and easy-to-use interface through both CLI and web application. The CLI should offer a clear menu-driven structure, while the web interface should provide a visually organized layout.

Information should be displayed in a clear and readable format, including product details, cart summaries, and confirmation messages. The system should respond quickly to user actions, ensuring a smooth experience.

Error messages must be meaningful and help users understand and correct their mistakes. The CLI should support both numeric and text-based inputs for flexibility, while the web interface should allow intuitive interaction.

3.1.3 Environmental Requirements

Environmental requirements define the technical conditions and resources needed for the system to operate effectively. These include hardware, software, and operational dependencies.

The system requires a standard computer or laptop with minimum specifications such as 4GB RAM and a basic processor. Since the application is lightweight, it does not require high-end hardware.

The system should run on a compatible operating system such as Windows, Linux, or macOS. These platforms support Python and provide the necessary environment for executing the application.

Python (version 3.8 or above) must be installed, as it is the primary programming language used for both frontend (CLI) and backend development. Required libraries such as Flask and requests must also be installed using a package manager like pip.

A web browser is required to run the web-based interface. Modern browsers such as Chrome, Firefox, or Edge can be used to access the application. The system depends on a JSON file for data storage. This file must be available and contain valid data, as it stores product details and cart information.

The Flask backend server must be running on localhost (127.0.0.1:5000) to handle requests from both CLI and web interfaces. Without the server, the system will not function correctly. The system uses the HTTP protocol for communication between frontend and backend, requiring proper configuration of the local network environment.

Conclusion: The requirements outlined above provide a clear and complete understanding of the system's expected functionality, user expectations, and operating conditions. By addressing functional, user, and environmental aspects, the system ensures reliable performance, usability, and ease of implementation. These requirements serve as a strong foundation for designing and developing the E-commerce application.

3.2 Design and Architecture

The design and architecture of the E-commerce system define how different components are organized, how they interact, and how data flows throughout the application. A well-structured architecture ensures that the system is scalable, maintainable, and efficient. In this project, the system is designed using a **client-server model**, combined with **a** three-tier architecture **and a** modular design approach**,** reflecting real-world software engineering practices.

The backend is implemented using the Flask framework, which acts as the central processing unit of the system. It handles all incoming requests, processes them using defined business logic, and sends appropriate responses back to the client. This clear separation between frontend and backend improves modularity and allows independent development and testing of each component.

The system follows a three-tier architectural pattern, consisting of the presentation layer, application layer, and data layer. Each layer has a specific role and interacts with the others in a structured manner.

The presentation layer includes both the CLI and the web interface. It is responsible for user interaction, such as displaying menus, capturing input, and presenting results. The CLI uses a menu-driven approach, allowing users to perform operations by entering numeric choices or simple text commands. The web interface, on the other hand, provides a structured layout with interactive elements for easier navigation. This layer does not perform any business logic; instead, it forwards user requests to the backend through HTTP calls.

The application layer is implemented using Flask and contains the core business logic of the system. It processes user requests, performs validations, and executes operations such as retrieving product data, managing the shopping cart, and handling checkout. The backend exposes a set **of** RESTful API endpoints, each designed to perform a specific function. These endpoints handle HTTP methods such as GET (for retrieving data) and POST (for

sending data). The application layer ensures that all operations follow defined rules, such as checking stock availability and validating user inputs.

The data layer is responsible for storing and managing system data. In this project, a JSON file is used as a lightweight database to store product details and cart information. The backend reads from this file when data is requested and updates it when changes occur. Additionally, the web interface uses **local Storage** to store temporary client-side data such as cart items and session details. This reduces dependency on the server for repeated operations and improves performance.

The system follows a clear and structured data flow. When a user performs an action, such as viewing products or adding an item to the cart, the request is sent from the frontend to the backend. The backend processes the request, interacts with the JSON file to retrieve or update data, and sends a response back to the frontend. The frontend then displays the result to the user. This flow ensures efficient handling of operations and maintains data consistency

A key aspect of the system design is modularity. The application is divided into separate modules, each responsible for a specific function. For example, the CLI handles user interaction, the backend manages business logic, and the data layer handles storage. This separation of concerns improves code readability, simplifies debugging, and allows easy extension of the system. New features, such as user authentication or payment integration, can be added without significantly affecting existing components.

The system also incorporates input validation and error handling mechanisms. The backend validates all incoming requests to ensure that inputs are correct and within acceptable limits. For instance, it checks whether a product exists and whether sufficient stock is available before adding it to the cart. If an error occurs, the backend returns a descriptive message, which is displayed by the frontend. The CLI also handles connection errors, such as when the server is not running, ensuring a smooth user experience.

From a performance perspective, the system is optimized for small-scale usage. Since all components run on a single machine and communication occurs through localhost, response

times are minimal. The use of JSON for data storage ensures fast read and write operations. However, this approach has limitations in terms of scalability, as it does not support concurrent access or large datasets.

The architecture also considers scalability and future enhancements. Although the current system is designed for a single-user environment, it can be extended to support multiple users by replacing the JSON file with a database and deploying the backend on a remote server. Similarly, the CLI can be replaced or complemented with advanced graphical interfaces. The use of RESTful APIs ensures that new clients can be integrated without major changes to the backend.

Security considerations in this system are minimal due to its local execution environment. The backend server runs on localhost, preventing external access. Basic input validation is implemented to avoid invalid operations. However, advanced security features such as authentication and encryption can be added in future versions.

In conclusion, the design and architecture of the E-commerce system provide a structured and efficient framework for implementing core e-commerce functionalities. By combining a client-server model, three-tier architecture, and modular design principles, the system achieves a balance between simplicity and functionality. It not only demonstrates key software engineering concepts but also provides a strong foundation for future development and scalability.

3.3 Implementation

The implementation phase of the E-commerce system focuses on converting the system design into a fully functional application using appropriate technologies and programming practices. This phase involves developing the frontend interfaces, backend services, and data handling mechanisms, and ensuring proper integration between all components. The system is implemented using Pytho**n**, with a clear separation between the CLI frontend, web frontend, and Flask backend, ensuring modularity and maintainability.

The system is divided into three main components: the CLI frontend, the web-based frontend, and the backend server. Each component is implemented independently but communicates with the others through well-defined interfaces. This separation allows for easier development, testing, and future enhancements.

The Flask backend serves as the core of the system and is responsible for handling all business logic and data processing. It is typically implemented in a Python file (such as *app.py*), where various API endpoints are defined. These endpoints correspond to different system functionalities, such as retrieving product data, managing the shopping cart, and processing checkout operations. Flask is used to create RESTful APIs that handle HTTP requests and return responses in JSON format.

The backend includes helper functions for data management. A data loading function reads product and cart information from a JSON file and converts it into a Python data structure, such as a dictionary. A data saving function writes updated data back to the JSON file whenever changes occur. These operations ensure data persistence across system usage.

One of the key API endpoints implemented is the product retrieval endpoint, which handles GET requests and returns a list of available products. Each product includes attributes such as name, price, and stock quantity. This endpoint is used by both the CLI and web frontend to display product information.

Another important endpoint is the cart management endpoint, which allows users to add items to the cart and view cart contents. When adding an item, the backend validates the input by checking whether the product exists and whether sufficient stock is available. If the validation is successful, the system updates the cart and reduces the stock quantity accordingly. If not, an error message is returned.

The checkout endpoint is responsible for finalizing the purchase. It verifies that the cart is not empty and then clears it after successful checkout. A confirmation message is sent back to the client, indicating that the order has been placed successfully.

The CLI frontend is implemented in a separate Python file (such as *main.py*) and acts as one of the user interaction layers. It provides a menu-driven interface where users can select options such as viewing products, adding items to the cart, viewing the cart, and performing checkout. The CLI captures user input and sends appropriate HTTP requests to the backend using the Python *requests* library.

The CLI implementation includes several functions to handle different operations. A menu display function presents available options in a structured format. An input handling function captures and validates user input. An action parsing function interprets both numeric selections and simple text commands, allowing flexibility in interaction. This enables users to interact with the system using commands such as “show products” or “add item,” in addition to numeric choices.

The web frontend is implemented as a simple Single Page Application (SPA) using HTML, CSS, and JavaScript. It provides a graphical interface for users to interact with the system. The web interface displays product lists, allows users to add items to the cart, and supports checkout functionality. JavaScript is used to handle user actions and send HTTP requests to the backend using the Fetch API.

The web frontend also uses local Storage to store temporary data such as cart items and session details. This reduces the number of server requests and improves performance. The stored data is synchronized with the backend when necessary, ensuring consistency.

Integration between the frontend and backend is achieved through RESTful API communication. Both the CLI and web frontend send HTTP requests to the Flask server and receive responses in JSON format. The system follows a stateless communication model, where each request contains all the information required for processing. This simplifies the implementation and improves scalability.

Error handling is implemented in both the frontend and backend. The backend validates all incoming requests and returns descriptive error messages for invalid inputs, such as incorrect product names or insufficient stock. The CLI handles connection errors using try-except blocks and displays user-friendly messages if the server is not accessible. The web frontend also handles errors by displaying alerts or messages to the user.

Input validation is a critical part of the implementation. The system ensures that all inputs are valid before processing them. For example, the quantity entered by the user must be a positive integer and should not exceed available stock. These validations prevent incorrect operations and maintain data integrity.

The project is organized into a clear file structure, separating backend, frontend, and data components. The backend file contains API logic, the CLI file handles command-line interaction, and the web files manage the graphical interface. The JSON file is used for data storage. This organization improves readability and makes the code easier to manage.

From a performance perspective, the system is optimized for small-scale usage. Since it runs on a local machine and uses lightweight technologies, response times are fast. The use of JSON for data storage ensures quick read and write operations, although it may not scale well for large applications.

The implementation is designed to be extensible. New features such as user authentication, payment integration, or database support can be added without major changes to the existing structure. The modular design and use of RESTful APIs make it easy to expand the system.

In conclusion, the implementation of the E-commerce system successfully translates the design into a working application. By integrating the CLI frontend, web frontend, Flask backend, and JSON data storage, the system achieves its intended functionality while maintaining simplicity and efficiency. The modular structure, validation mechanisms, and error handling ensure that the application is reliable, maintainable, and ready for further development.

3.4 Testing

Testing is a crucial phase in the software development lifecycle, aimed at verifying that the system functions correctly, meets specified requirements, and is free from major defects. In the E-commerce system, testing ensures that all components, including the CLI frontend, web frontend, Flask backend, and data storage layer, work together seamlessly. The primary objective is to improve system reliability, performance, and user experience.

The testing process in this project includes multiple levels, such as unit testing, integration testing, system testing, and user acceptance testing. Each level focuses on different aspects of the system to ensure overall quality.

Unit testing involves testing individual components or functions independently. In this system, unit testing is applied to backend functions such as loading and saving JSON data, validating inputs, and handling API requests. For example, functions that check whether a product exists or whether the quantity entered is valid are tested separately to ensure they produce correct results. Similarly, frontend functions for handling input and displaying output are tested to verify proper behavior.

Integration testing focuses on verifying the interaction between different components. In this project, it ensures that the CLI and web frontends communicate correctly with the Flask backend through RESTful APIs. For example, when a user requests to view products, the frontend sends an HTTP request, and the backend should return accurate data in JSON format. Integration testing also checks whether the backend correctly interacts with the JSON file for data retrieval and updates.

System testing evaluates the complete application as a whole. It involves testing all functionalities together to ensure they work as expected. Key operations such as viewing products, adding items to the cart, viewing cart contents, and performing checkout are tested in sequence. For instance, a typical test scenario includes adding multiple products to the cart and completing checkout. The system should correctly update stock values, display accurate cart information, and generate a confirmation message.

User acceptance testing (UAT) is conducted to evaluate the system from the user's perspective. Users interact with both the CLI and web interfaces to perform common tasks. Feedback is collected to assess usability, clarity, and overall experience. This helps identify areas where the system can be improved, such as simplifying navigation or improving output formatting.

The system includes several test cases to verify functionality. For example, one test case checks whether the product list is displayed correctly. Another verifies that adding a valid product updates the cart successfully. Additional test cases handle error scenarios, such as entering an invalid product name or requesting a quantity greater than available stock. These tests ensure that the system behaves correctly in both normal and exceptional conditions.

Error handling testing is an important part of the process. The system is tested to ensure that it responds appropriately to invalid inputs and unexpected situations. For instance, if the backend server is not running, the CLI should display a connection error message. Similarly, if incorrect data is provided, the backend should return a clear and descriptive error message without crashing.

Performance testing is conducted at a basic level to ensure that the system responds quickly to user actions. Since the application runs locally and uses lightweight technologies, operations such as retrieving products and updating the cart are expected to execute with minimal delay. Testing confirms that the system performs efficiently under normal usage conditions.

Data validation testing ensures that all inputs are properly checked before processing. The system verifies that quantities are positive integers and do not exceed available stock. This prevents incorrect data from entering the system and maintains data integrity.

Another important aspect is data consistency testing. The system is tested to ensure that operations correctly update the stored data. For example, when an item is added to the cart, the stock quantity should decrease accordingly. After checkout, the cart should be cleared. These checks ensure that the system maintains a consistent state.

Testing is supported by a structured approach, where test cases are defined with input conditions, expected outputs, and actual results. This helps in identifying issues and tracking improvements. Both manual testing and basic debugging techniques are used during development.

Despite thorough testing, minor issues may still exist. However, the testing process significantly reduces errors and ensures that the system is stable and reliable. The results show that the system meets its functional requirements and performs effectively for small-scale usage.

In conclusion, testing plays a vital role in ensuring the quality and reliability of the E-commerce system. By applying different levels of testing and covering various scenarios, the system achieves a high level of accuracy and usability. This process ensures that the application is ready for use and provides a solid foundation for future improvements.

3.5 Graphical User Interface (GUI) Layout

Although the E-commerce system is implemented using a Command-Line Interface (CLI), the concept of interface layout remains essential for ensuring smooth user interaction. In the High-Level Design (HLD), the CLI represents the presentation layer, while the Low-Level Design (LLD) defines how inputs are processed and mapped to backend operations. Even without graphical components, the system is carefully structured to provide a clear, consistent, and user-friendly interaction model.

The CLI is designed to simulate a structured interface similar to a graphical system by using organized text output. When the user launches the application, a main menu is displayed, acting as the central navigation hub. This menu includes options such as viewing products, adding items to the cart, viewing cart details, performing checkout, and exiting the system. The use of numbered options ensures quick and error-free navigation, especially for users who prefer structured interaction.

To enhance usability, the system supports both numeric and text-based inputs. This feature, defined in the LLD, allows users to interact using commands such as “show products” or “view cart,” in addition to numeric selections. The input parsing mechanism interprets these commands and maps them to corresponding backend API calls. This hybrid interaction approach improves flexibility and makes the system more intuitive.

The product display layout is designed to present information retrieved from the backend in a readable and structured format. Each product is displayed with attributes such as name, price, stock availability, and specifications. Proper spacing, alignment, and formatting techniques are used to ensure clarity. This reflects the structured data flow from the backend, where product details are stored in a JSON file and served via API endpoints.

The cart interface provides a summarized view of selected items. It displays product names, quantities, individual prices, and total cost. This layout helps users review their selections before proceeding to checkout. The checkout interface includes a confirmation step, ensuring

that users verify their decision before finalizing the purchase. After successful checkout, a confirmation message is displayed, indicating completion of the transaction.

Error handling plays a significant role in the interface design. The system displays clear and consistent messages when invalid inputs are entered or when backend validation fails. For example, if a user attempts to add a product that does not exist or requests a quantity beyond available stock, the system provides a descriptive error message. This behavior ensures that users are guided correctly without confusion.

The interface design also follows key usability principles such as consistency, simplicity, and feedback. All menus and outputs follow a uniform structure, making it easier for users to understand the system. Feedback messages confirm user actions, improving confidence and interaction quality.

Although the system is text-based, it can be extended into a graphical interface in the future. Since the backend is built using RESTful APIs, a web or mobile interface can replace the CLI without changing the core logic. This highlights the flexibility of the system architecture.

However, the CLI has certain limitations. It lacks visual elements such as images and interactive components, which may make it less appealing to non-technical users. Despite this, it remains efficient, lightweight, and suitable for development and testing environments.

In conclusion, the GUI layout of the system, though implemented as a CLI, effectively fulfills the role of a presentation layer. It provides structured navigation, clear output, and flexible interaction, aligning with both HLD and LLD design principles.

3.6 Customer Testing

Customer testing, also known as User Acceptance Testing (UAT), is conducted to evaluate the system from the user's perspective. This phase ensures that the application meets user expectations and performs all required functions as defined in the HLD and LLD.

In this project, customer testing involves users interacting with the CLI system to perform typical e-commerce operations. These operations include viewing products, adding items to the cart, checking cart contents, and completing checkout. Each of these actions corresponds to backend API endpoints and workflow logic defined in the system design.

The testing process begins by introducing users to the system and explaining its functionalities. Users are then asked to perform specific tasks while their interactions are observed. This helps identify usability issues and areas where users may face difficulty. For example, if users struggle to understand menu options or input commands incorrectly, it indicates a need for interface improvement.

One of the main focus areas of customer testing is ease of use. The CLI is designed to be simple and intuitive, but testing verifies whether users can navigate it effectively. Feedback helps determine whether the menu structure is clear and whether instructions are easy to follow.

Another important aspect is input validation. The LLD defines how inputs are processed and validated. During testing, users provide both valid and invalid inputs to ensure the system responds appropriately. For instance, entering a negative quantity or an invalid product name should trigger an error message. This confirms the robustness of validation mechanisms.

System responsiveness is also evaluated. Since the application uses localhost communication, responses are expected to be fast. Testing ensures that API requests are processed efficiently and results are displayed without delay. This contributes to a smooth user experience.

Data consistency is another key factor. When users add items to the cart, the backend updates the JSON file, and changes are reflected immediately. Users verify that cart contents and product stock are updated correctly after each operation. This ensures synchronization between system components.

Customer feedback plays a crucial role in improving the system. Many users find the system easy to use due to its structured menu and flexible input options. However, some users suggest enhancements such as adding a graphical interface, improving output formatting, and including advanced features like user authentication or payment integration.

Testing also evaluates error handling. Users intentionally perform incorrect actions to observe how the system responds. Effective error messages help users understand issues and correct their inputs. This improves overall usability and reliability.

In conclusion, customer testing confirms that the system meets its intended objectives. It validates the interaction between the CLI, backend, and data storage layers, ensuring that the system works as designed. The feedback collected provides valuable insights for future improvements and feature expansion.

3.7 Evaluation

Evaluation is a critical phase in the software development lifecycle that focuses on assessing the overall effectiveness, performance, and quality of the system. In the context of the E-commerce CLI system, evaluation is conducted based on how well the system meets its objectives, how efficiently it performs its operations, and how closely it aligns with the High-Level Design (HLD) and Low-Level Design (LLD). This phase helps identify both strengths and limitations, providing a clear understanding of the system's capabilities and areas for future improvement.

From a functional perspective, the system successfully implements all core features of a basic e-commerce application. Users are able to view products, add items to a cart, review cart contents, and complete the checkout process. These functionalities are implemented through RESTful API endpoints in the backend and are accessed via the CLI interface. The separation of concerns defined in the HLD—dividing the system into presentation, application, and data layers—ensures that each component performs its role effectively. The LLD further supports this by clearly defining the behaviour of each module, including input handling, request processing, and data manipulation.

Usability is another key aspect evaluated in this system. The CLI interface is designed to be simple, structured, and easy to navigate. The menu-driven approach provides clear guidance to users, allowing them to perform operations without confusion. Additionally, the support for both numeric and text-based inputs enhance flexibility and user convenience. Feedback obtained during customer testing indicates that users find the system intuitive and easy to use, even without prior technical knowledge. However, it is also observed that non-technical users may prefer a graphical interface, which highlights a potential area for enhancement.

Performance efficiency is evaluated based on the system's response time and resource utilization. Since the application operates on a single machine and uses localhost communication between the CLI and backend, it achieves fast response times with minimal latency. The use of a JSON file for data storage allows quick read and write operations, contributing to overall efficiency. The system performs well under normal conditions and handles user requests promptly. However, the evaluation also identifies that this approach may not scale effectively for larger

datasets or multi-user environments, as JSON-based storage lacks advanced features such as indexing and concurrency control.

Reliability is an important factor in evaluating system quality. The system includes robust input validation and error handling mechanisms to ensure stable operation. The backend validates all incoming requests, checking for conditions such as valid product names, appropriate quantities, and stock availability. When errors occur, the system provides clear and descriptive messages, helping users understand and correct their inputs. The CLI also handles connection-related issues, such as when the backend server is not running, by displaying appropriate error messages. These features contribute to the overall reliability and robustness of the system.

Maintainability is another strength of the system, as defined by its modular architecture. The separation of the CLI frontend, Flask backend, and JSON data layer ensures that each component can be developed, tested, and modified independently. The LLD further enhances maintainability by defining clear functions and responsibilities for each module. This structured approach makes it easier for developers to debug the system, add new features, or make changes without affecting other components. The use of Python, known for its readability and simplicity, also contributes to maintainable code.

Scalability is evaluated to determine how well the system can handle growth in terms of users, data, and functionality. While the current implementation is suitable for a single-user environment, it has limitations when scaling to larger applications. The use of a JSON file as the data storage mechanism restricts the system's ability to handle concurrent access and large volumes of data. Additionally, the lack of session management and multi-user support limits its applicability in real-world scenarios. However, the architecture defined in the HLD is flexible enough to support future enhancements, such as replacing the JSON file with a database and deploying the backend on a remote server.

Security considerations in this system are minimal due to its local execution environment. The backend server runs on localhost, preventing external access and reducing exposure to security threats. Basic input validation is implemented to prevent invalid operations. However, advanced security features such as user authentication, authorization, data encryption, and secure

communication protocols are not included in this version. These features would be essential for a production-level system and represent an important area for future development.

Another important aspect of evaluation is data consistency and integrity. The system ensures that all operations maintain a consistent state. For example, when a product is added to the cart, the stock quantity is reduced accordingly, and the updated data is saved in the JSON file. Similarly, after checkout, the cart is cleared, and the system reflects the updated state. This consistent data flow between the CLI, backend, and storage layer demonstrates proper synchronization as defined in the system design.

User satisfaction is also considered during evaluation. Feedback collected from users indicates that the system is efficient, easy to use, and responsive. Users appreciate the simplicity of the CLI interface and the ability to perform tasks quickly. However, some users suggest improvements such as adding more features, enhancing output formatting, and introducing a graphical interface. These suggestions provide valuable insights for future enhancements and indicate areas where the system can be improved.

Despite its strengths, the system has certain limitations. The absence of a graphical interface may reduce its appeal to non-technical users. The reliance on a JSON file for data storage limits scalability and performance in larger applications. Additionally, the system does not include advanced features such as user accounts, payment processing, or order tracking, which are commonly found in modern e-commerce platforms. These limitations highlight the fact that the system is primarily designed for educational purposes rather than real-world deployment.

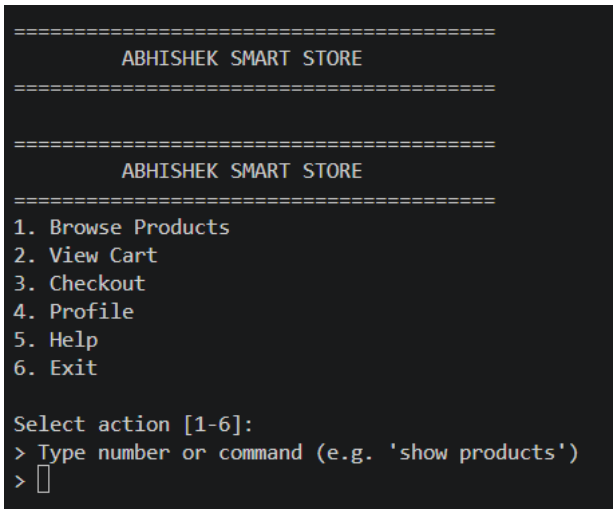
However, the system successfully achieves its primary objective of demonstrating key software engineering concepts. It provides a clear understanding of how a client-server architecture operates, how RESTful APIs enable communication between components, and how data is managed and persisted. The modular design and separation of concerns reflect real-world application development practices, making the system a valuable learning tool.

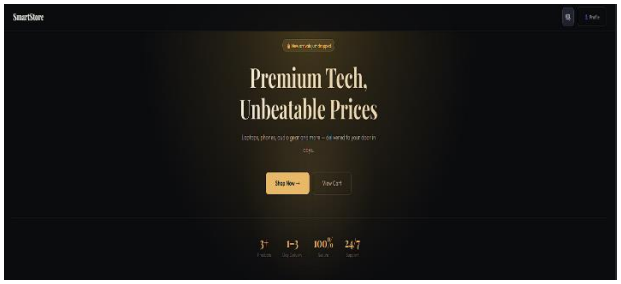
In conclusion, the evaluation of the E-commerce CLI system shows that it is a functional, efficient, and well-structured application that meets its intended objectives. It performs all required operations effectively and provides a positive user experience within its scope. While it has limitations in terms of scalability, security, and advanced features, it establishes a strong foundation for future development. The insights gained from this evaluation can guide further improvements, enabling the system to evolve into a more robust and scalable application.

4. Snapshots of the project

The snapshots presented in this section illustrate the working of the E-commerce CLI system. These screenshots provide a visual representation of how users interact with the application through the command-line interface. Each snapshot highlights a specific feature of the system, including product browsing, cart management, checkout process, and other functionalities. These visuals help in understanding the workflow and usability of the system.

Figure 4.1: Main Menu Interface



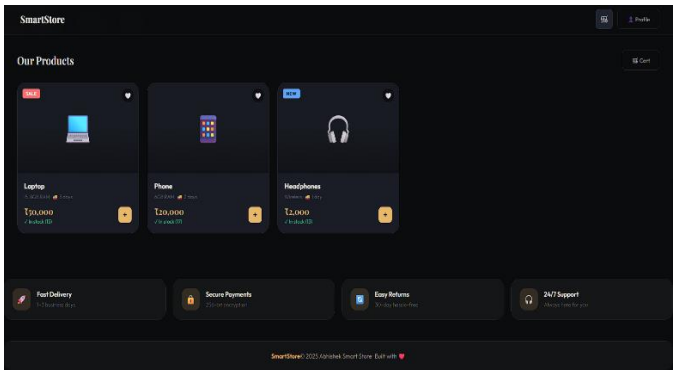


Description:

The above snapshot shows the main menu of the system. It provides users with various options such as browsing products, viewing the cart, proceeding to checkout, accessing profile details, viewing help instructions, and exiting the application. This menu acts as the central navigation point of the system and ensures easy user interaction.

Figure 4.2: Browse Product

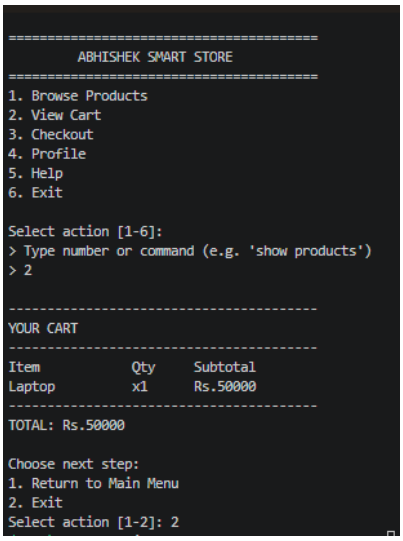
```
Select action [1-6]:
> Type number or command (e.g. 'show products')
> 1

----------------------------------------
PRODUCT LIST
----------------------------------------
ID  Name          Price     Stock
1   Laptop        Rs.50000  15
2   Phone         Rs.20000  17
3   Headphones    Rs.2000   13
----------------------------------------
Select product (ID/name) or 'back': 1

----------------------------------------
PRODUCT DETAILS
----------------------------------------
Name     : Laptop
Price    : Rs.50000
Stock    : 15
Delivery : 3 days
Specs    : i5, 8GB RAM
----------------------------------------
1. Add to Cart
2. Back to Product List
Select action [1-2]: 1
Enter quantity: 1

Confirm add to cart:
Item     : Laptop
Quantity : 1
1. Confirm
2. Cancel
Select action [1-2]: 1
[OK] added to cart

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]: 1
```

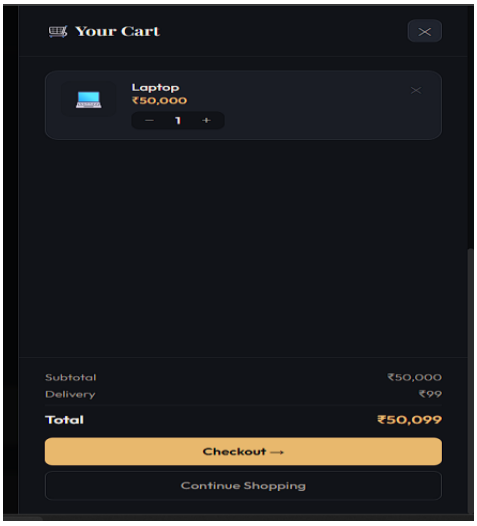


Description:

This snapshot displays the product browsing feature of the system. Users can view the list of available products along with details such as price, stock availability, and specifications. This feature allows users to explore products before making a selection.

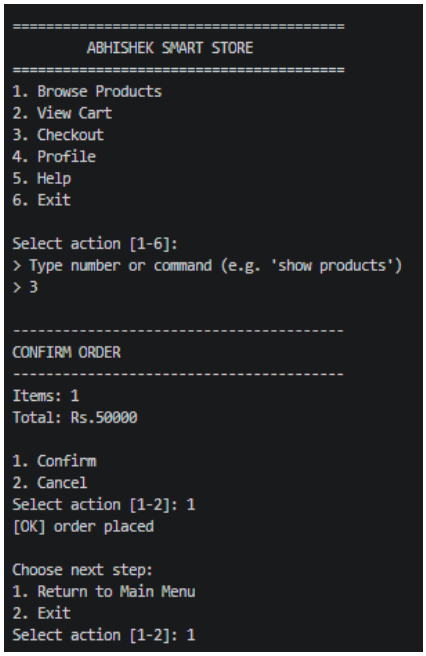
Figure 4.3: View Cart

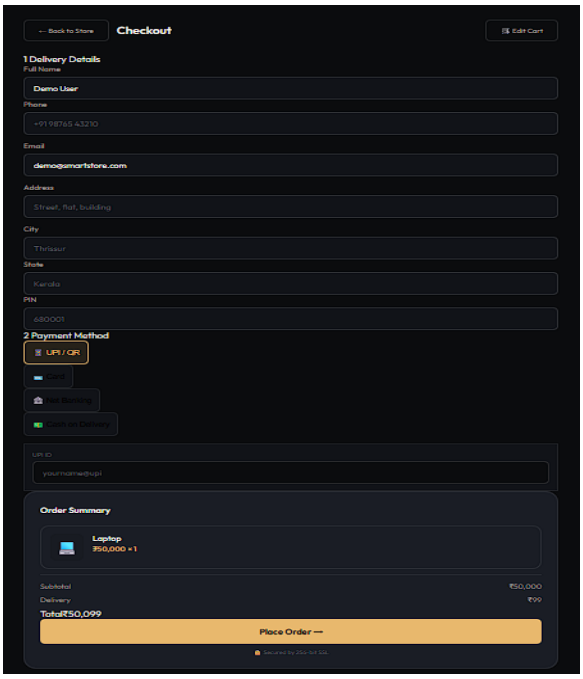




Description: The above snapshot shows the contents of the shopping cart. It includes the list of selected products along with their quantities and total cost. This feature helps users review their selected items before proceeding to checkout.

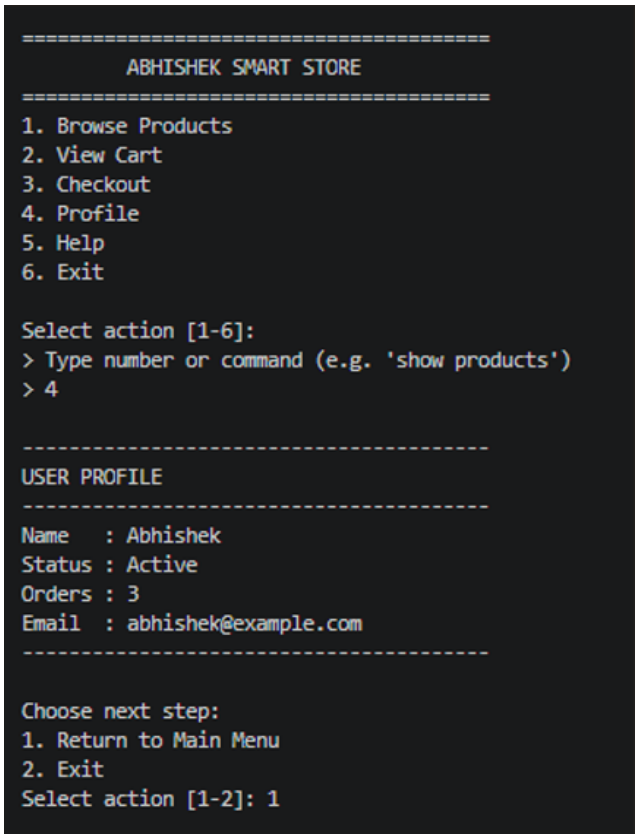
Figure 4.4: Checkout Process

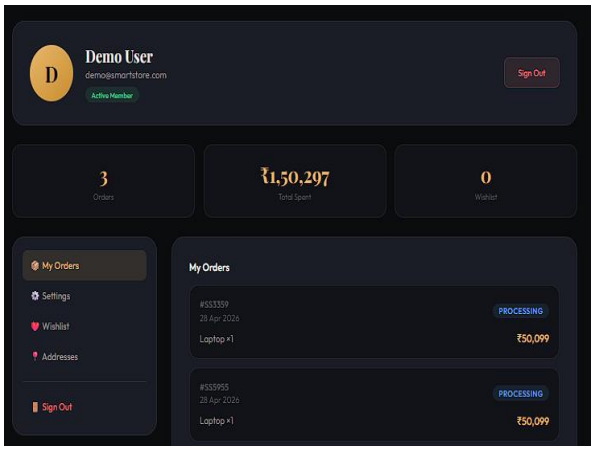




Description: This snapshot represents the checkout process of the system. Users confirm their purchase, and the system processes the order. A confirmation message is displayed upon successful completion of the transaction.

Figure 4.5: User Profile





Description: The above snapshot shows the user profile section. It displays user-related information such as name, email, or other details. This feature helps personalize the user experience and manage user-specific data.

Figure 4.6: Help Section

```
========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 5

----------------------------------------
HELP
----------------------------------------
Actions:
- show products / list / view items / 1
- cart / 2
- checkout / 3
- profile / 4
- help / 5
- exit / 6
----------------------------------------

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]:
```

Description: This snapshot shows the help section of the system. It provides guidance to users on how to use the application, including available commands and navigation instructions. This feature improves usability, especially for new users.

Figure 4.7: Exit Option

```
========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 6
Exiting Abhishek Smart Store.
```

Description:
The above snapshot shows the exit functionality of the system. It allows users to safely close the application. This ensures proper termination of the program.

5. Conclusion

The E-commerce CLI system developed in this project successfully demonstrates the fundamental concepts of software development, system design, and application architecture through a simplified yet functional implementation. The primary objective of this project was to design and develop a command-line-based e-commerce platform that enables users to browse products, manage a shopping cart, and complete a checkout process. This objective has been achieved effectively by integrating a CLI-based frontend, a Flask-powered backend, and a JSON-based data storage mechanism.

One of the major achievements of this project is the successful implementation of a client-server architecture, as defined in the High-Level Design (HLD). The system clearly separates the presentation layer (CLI), application layer (Flask backend), and data layer (JSON storage), ensuring a well-organized and modular structure. This separation of concerns improves maintainability, scalability, and ease of development. The Low-Level Design (LLD) further strengthens this by defining the internal working of each module, including input handling, API communication, and data processing.

The system effectively implements the core functionalities of an e-commerce application. Users can view available products, add items to their cart, review cart contents, and complete the checkout process. These operations are executed through RESTful APIs, which facilitate structured communication between the CLI and backend using HTTP requests and JSON responses. This approach not only ensures efficient data exchange but also makes the system extensible for future enhancements, such as integrating web or mobile interfaces.

Another important aspect of the project is the design of the CLI interface. Despite being text-based, the interface is structured, consistent, and user-friendly. The menu-driven approach, along with support for both numeric and text-based inputs, enhances usability and flexibility. Clear prompts, formatted outputs, and descriptive error messages

contribute to a smooth user experience. This demonstrates that even a non-graphical interface can be effectively designed to support user interaction.

From a performance standpoint, the system is lightweight and efficient. Since all components run on a local machine and communication occurs through the localhost interface, response times are fast and reliable. The use of a JSON file for data storage simplifies implementation and ensures data persistence. However, this approach is best suited for small-scale applications and serves as a learning tool for understanding data handling concepts.

The project also highlights several important software engineering practices, including modular design, input validation, error handling, and structured data flow. Each component of the system is designed to perform a specific function, making the overall system easier to understand, test, and maintain. The implementation of validation checks ensures data accuracy, while error handling mechanisms improve system reliability and robustness.

Despite its strengths, the system has certain limitations. The use of a JSON file for data storage restricts scalability and does not support concurrent access by multiple users. The absence of advanced features such as user authentication, payment processing, and order tracking limits its applicability in real-world scenarios. Additionally, the CLI interface, while efficient, may not be as intuitive for non-technical users compared to graphical user interfaces.

However, these limitations do not reduce the value of the project. Instead, they highlight opportunities for future improvements. The system can be enhanced by integrating a database for better data management, implementing user authentication for security, and developing a graphical interface for improved usability. The existing architecture, based on RESTful APIs and modular design, provides a strong foundation for these enhancements.

In conclusion, the E-commerce CLI system successfully meets its intended objectives and demonstrates key concepts of modern software development. It provides a practical understanding of how different components interact within a system and how real-world applications are structured. The project serves as an effective learning platform and a solid base for future development. With further enhancements, it can evolve into a more advanced and scalable e-commerce solution.

6. Further development or research

The E-commerce CLI system developed in this project provides a simple and functional platform for performing basic online shopping operations. While the system successfully demonstrates key concepts such as client-server architecture, RESTful API communication, and modular design, there are several opportunities for further development and research. These improvements can enhance the system's functionality, usability, scalability, and overall effectiveness, making it closer to a real-world e-commerce application.

One of the most important areas for further development is the integration of a database management system such as MySQL, PostgreSQL, or MongoDB. In the current implementation, the system uses a JSON file to store product and cart data. While this approach is simple and easy to implement, it has limitations in terms of scalability, data consistency, and concurrent access. A database system would allow efficient storage and retrieval of large volumes of data and support multiple users simultaneously. It would also provide advanced features such as indexing, transactions, and query optimization, which improve system performance and reliability. Research can be conducted on selecting the most suitable database type (relational or NoSQL) based on system requirements.

Another significant enhancement is the development of a Graphical User Interface (GUI). The existing command-line interface, although efficient and lightweight, may not be suitable for all users, particularly those without technical expertise. A GUI would provide a more interactive and visually appealing experience, making the system easier to use. This can be implemented using web technologies such as HTML, CSS, and JavaScript, or frameworks like React or Angular. Desktop-based solutions can also be explored. Since the backend already uses RESTful APIs, research can focus on how different frontend technologies can be integrated seamlessly with the existing architecture.

The implementation of user authentication and authorization is another important area for future development. Currently, the system operates as a single-user application without any form of identity management. Adding features such as user registration, login, and profile management would allow multiple users to interact with the system securely. Authentication mechanisms such as session-based authentication or token-based systems (e.g., JWT) can be studied and

implemented. Authorization can further control access to different functionalities based on user roles. This enhancement would improve both security and personalization.

Integrating a payment gateway is another area that can significantly improve the system. At present, the checkout process only simulates a purchase without handling actual transactions. By integrating digital payment methods such as credit/debit cards or online payment services, the system can provide a more realistic shopping experience. This would involve researching secure payment processing techniques, API integration, and data protection measures. It also introduces important concepts related to financial transactions and security.

The system can also be extended by implementing features such as order history and order tracking. Currently, once a purchase is completed, the cart is cleared without maintaining any record of the transaction. By storing order details, users can review their past purchases and track the status of their orders. This feature enhances user experience and aligns the system with real-world e-commerce platforms.

Another area for enhancement is the implementation of advanced product search and filtering capabilities. Instead of displaying products in a simple list, the system can allow users to search for items using keywords, categories, or price ranges. Research can also be conducted on recommendation systems, which suggest products based on user behaviour or preferences. This would improve usability and provide a more personalized experience.

Multi-user support is another important aspect for future development. The current system is designed for a single-user environment, which limits its scalability. By modifying the architecture to support multiple users, the system can handle concurrent operations more effectively. This would require improvements in data management, session handling, and concurrency control, especially when combined with a database system.

Security is another critical area for further research. While the current system includes basic input validation, it lacks advanced security mechanisms. Future enhancements can include implementing HTTPS for secure communication, encrypting sensitive data, and protecting the

system against common vulnerabilities such as injection attacks. Research into secure coding practices and authentication protocols would help improve system reliability and safety.

Performance optimization is also an important consideration. Although the system performs efficiently in a local environment, further improvements can be made by introducing caching mechanisms, optimizing database queries, and using asynchronous processing. These techniques can significantly improve system performance, especially as the application grows in complexity.

Additionally, logging and monitoring features can be implemented to track system activity and performance. Logging helps in debugging and identifying issues, while monitoring tools provide insights into system behaviour. Research can be conducted on tools and frameworks that support logging and performance analysis.

From a research perspective, the system can also be explored as a foundation for more advanced architectural patterns, such as microservices. Instead of a single backend application, the system can be divided into smaller, independent services for product management, user management, and order processing. This approach improves scalability, flexibility, and maintainability, and is widely used in modern software systems.

In conclusion, the E-commerce CLI system offers a strong foundation for further development and research. By integrating a database, developing a graphical interface, implementing authentication and payment systems, and enhancing scalability and security, the system can evolve into a more advanced and practical application. These improvements not only enhance the functionality of the system but also provide valuable opportunities to explore advanced concepts in software engineering and system design.

7.Reference

Flask Official Documentation - https://flask.palletsprojects.com/. Python Requests Library Documentation.

JSON File Format Specification. REST API Design Best Practices.

Command-Line Interface Design Patterns. E-commerce System Architecture Principles. Roy Fielding Dissertation on REST Architecture.

Python Official Documentation - https://docs.python.org/3/.

Mozilla Developer Network — Fetch API Documentation: https://developer.mozilla.org/enUS/docs/Web/API/Fetch_API.

Web Storage API Specification — localStorage/sessionStorage. Flask Template Rendering (Jinja2) Documentation. Single-Page Application Architecture Patterns and Best Practices.

CSS Custom Properties (Variables) — W3C Specification.

8. Appendix

Complete Source File List:

app.py

templates/index.html

static/style.css

static/app.js

data. Json

main.py (optional CLI)

API Endpoints

GET /products – Retrieve product list

GET /cart – View cart items

POST /add-to-cart – Add product to cart

POST /checkout – Complete purchase

Test Case Examples

Test Case	Input	Expected Output
View Products	Option 1	Display product list
Add Valid Item	Laptop, Qty: 2	Added to cart
Add Invalid Item	Wrong name	Error message
Checkout Empty Cart	Option 4	Warning message

Error Messages

Examples of system-generated messages:

Invalid input. Please try again.

Product not found.

Insufficient stock available.

Cart is empty. Cannot proceed to checkout.

Abbreviations

CLI – Command Line Interface

API – Application Programming Interface

JSON – JavaScript Object Notation

9.Weekly Report

9.1 Week 1

26-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1 For week commencing 25 March 2026 Programme: BCA (AI,CLOUD COMPUTING &DEVOPS)

Student Name: Abhishek k.k Register Number: **23BCAICD007**
WPPR: 1
Internal Guide's Name: MS.Yuktha
MAJOR PROJECT Title:

Natural Language Interface for E-Commerce Infrastructure

Targets set for the week:

- Understand project requirements and scope
- Finalize project title and objectives
- Study basics of NLP and chatbot systems
- Design overall system flow

Progress/Achievements for the week:

- Clearly defined project scope (CLI-based chatbot only)
- Understood keyword-based NLP approach
- Prepared project documentation (overview, objective, scope)
- Designed basic architecture (user → chatbot → parser → JSON data)

26-03-2026

Future Work Plans:

- Start implementation of CLI interface
- Create sample product dataset (JSON)
- Begin command parsing logic

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

9.2 Week 2

05-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 2 For week commencing 25 March 2026 Programme: BCA (AI,CLOUD COMPUTING &DEVOPS)

Student Name: Abhishek k.k
Register Number: **23BCAICD007**
WPPR: 2
Internal Guide's Name: MS.Yuktha
MAJOR PROJECT Title:

Natural Language Interface for E-Commerce Infrastructure

Targets set for the week:

- Develop CLI chatbot interface
- Create JSON files for products and cart
- Implement basic input/output handling

Progress/Achievements for the week:

- Built initial CLI chatbot using Python
- Created products.json and cart.json
- Implemented user input handling loop
- Displayed product list from JSON

26-03-2026

Future Work Plans:

- Implement keyword-based command recognition
- Add "add to cart" functionality
- Improve chatbot responses

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

9.3 Week 3

13-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 3 For week commencing 25 March 2026 Programme: BCA (AI,CLOUD COMPUTING &DEVOPS)

Student Name: Abhishek k.k
Register Number: **23BCAICD007**
WPPR: 3
Internal Guide's Name: MS.Yuktha
MAJOR PROJECT Title:

Natural Language Interface for E-Commerce Infrastructure

Targets set for the week:

- Implement NLP keyword parsing
- Add core e-commerce functionalities
- Test chatbot responses

Progress/Achievements for the week:

- Implemented keyword-based intent detection
- Added features:
- Show products
- Add to cart
- View cart
- Parsed user commands like "buy", "show", "cart"
- Ensured correct data updates in JSON

26-03-2026

Future Work Plans:

- Implement checkout functionality
- Improve error handling
- Refine chatbot interaction flow

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

9.4 Week 4

21-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 4 For week commencing 25 March 2026 Programme: BCA (AI,CLOUD COMPUTING &DEVOPS)

Student Name: Abhishek k.k
Register Number: **23BCAICD007**
WPPR: 4
Internal Guide's Name: MS.Yuktha
MAJOR PROJECT Title:

Natural Language Interface for E-Commerce Infrastructure

Targets set for the week:

- Complete remaining features
- Test full system
- Prepare final documentation

Progress/Achievements for the week:

- Implemented checkout feature
- Completed full chatbot workflow
- Tested all commands and scenarios
- Finalized project documentation
- Fixed bugs and improved usability

26-03-2026

Future Work Plans:

- Prepare for viva presentation
- Demonstrate working system
- Minor improvements if required

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**